

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИТМО  
ITMO University  
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №6  
по дисциплине “Web-программирование”

Выполнил: студент  
группы К3323  
Влазнев Д. В.  
Преподаватель:  
Марченко Е. В.

г. Санкт-Петербург,  
2024

## Содержание

|                    |    |
|--------------------|----|
| Содержание.....    | 2  |
| Введение.....      | 3  |
| Упражнение 1 ..... | 4  |
| Упражнение 2 ..... | 5  |
| Упражнение 3 ..... | 6  |
| Упражнение 4 ..... | 8  |
| Вывод.....         | 10 |

## **Введение**

**Цель работы:** изучить работу клиент-серверных приложений.

## Упражнение 1

Было сделано простое клиент-серверное приложение. На сервере был настроен сокет на TCP соединение. Сервер принимает сообщение, выводит его и отправляет сообщение клиенту. Клиент также с помощью сокета подключается к серверу. На рисунках 1–2 представлен код приложения.

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 12345))
server_socket.listen(1)

conn, addr = server_socket.accept()

data = conn.recv(1024)
print(f'Сообщение от клиента: {data.decode()}')

conn.sendall(b'Hello, client')

conn.close()
server_socket.close()
```

Рисунок 1 – Сервер

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))

client_socket.sendall(b'Hello, server')

data = client_socket.recv(1024)
print(f'Сообщение от сервера: {data.decode()}')

client_socket.close()
```

Рисунок 2 – Клиент

На рисунках 3–4 представлен вывод этих программ соответственно.

```
Сообщение от клиента: Hello, server
```

Рисунок 3 – Вывод сервера

```
Сообщение от сервера: Hello, client
```

Рисунок 4 – Вывод клиента

## Упражнение 2

В этом упражнении сервер принимает строку, состоящую из двух чисел, и вычисляет гипотенузу, после отправляет клиенту. На рисунках 5–6 представлен код приложения.

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('localhost', 12345))
s.listen(1)

def hip(a, b): 1 usage
    return float((a ** 2 + b ** 2) ** 0.5)

while True:
    client_socket, address = s.accept()
    try:
        data = client_socket.recv(1024)
        if data:
            decoded = data.decode('utf-8')
            pif_a, pif_b = map(float, decoded.split(' '))
            client_socket.send(f"{hip(pif_a, pif_b)}".encode('utf-8'))
    except KeyboardInterrupt:
        s.close()
```

Рисунок 5 – Сервер

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('localhost', 12345))

a = float(input("Enter the first number: "))
b = float(input("Enter the second number: "))
nums = f"{a} {b}"

s.send(nums.encode('utf-8'))

msg = s.recv(1024)
umsg = msg.decode('utf-8')
print(umsg)
s.close()
```

Рисунок 6 – Клиент

На рисунке 7 представлен вывод работы клиента.

```
Enter the first number: 3
Enter the second number: 4
5.0
```

Рисунок 7 – Вывод сервера

### Упражнение 3

В этом упражнении сервер отдает клиенту HTML страницу, хранящуюся в файле, и добавляет к нему нужные заголовки. В заголовках используется межстрочный разделитель `\r\n`, потому что это является стандартом HTTP 1.1. Также важно прочитать входящее сообщение от клиента, даже если оно не используется и тогда можно получить валидный HTML документ, используя браузер. На рисунках 8–9 представлен код приложения.

```
import socket

def read_html_file(file_name): 1 usage
    with open(file_name, 'r', encoding='utf-8') as file:
        return file.read()

def create_http_response(html_content): 1 usage
    response = (
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: {}\r\n"
        "Connection: close\r\n"
        "\r\n"
        "{}"
    ).format(*args: len(html_content), html_content)
    return response

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('localhost', 12345))
s.listen(1)

while True:
    client_socket, client_address = s.accept()
    with client_socket:
        request = client_socket.recv(1024)
        html_content = read_html_file('index.html')
        http_response = create_http_response(html_content)
        client_socket.sendall(http_response.encode('utf-8'))
```

Рисунок 8 – Сервер

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('localhost', 12345))

msg = s.recv(1024)
umsg = msg.decode('utf-8')
print(umsg)
s.close()
```

Рисунок 9 – Клиент

На рисунках 10–12 представлен HTML документ и выводы работы клиента и браузера.

```
<!DOCTYPE html>
<html lang="en">
<body>

<h1>Hello world!</h1>

</body>
</html>
```

Рисунок 10 – HTML документ

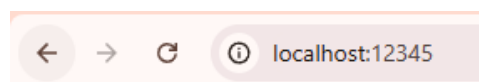
```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 80
Connection: close

<!DOCTYPE html>
<html lang="en">
<body>

<h1>Hello world!</h1>

</body>
</html>
```

Рисунок 11 – Вывод клиента



**Hello world!**

Рисунок 12 – Вывод браузера

## Упражнение 4

В этом упражнении был реализован многопользовательский чат. Сервер хранит список пользователей и отправляет сообщения одного пользователя всем остальным. На рисунке 13 представлен код функции сервера для добавления пользователя в чат. Пользователь вводит имя и если оно не занято, то присоединяется к чату. Остальные пользователи уведомляются об этом.

```
def try_add_user(client_socket, client_address): 1 usage
    print(f"Новое подключение {client_address}")

    client_socket.send("Введите свое имя: ".encode('utf-8'))
    user_name = client_socket.recv(1024).decode('utf-8')

    if user_name in users:
        client_socket.send("Имя уже занято, подключение прервано.\n".encode('utf-8'))
        client_socket.close()
        return
    users[user_name] = client_socket

    broadcast(f"{user_name} присоединился к чату".encode('utf-8'), client_socket)

    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')

            if not message:
                continue

            broadcast(f"{user_name}: {message}".encode('utf-8'), client_socket)
        except:
            client_socket.close()
            del users[user_name]
            broadcast(f"{user_name} покинул чат".encode('utf-8'))
```

Рисунок 13 – Функция сервера

Для каждого пользователя используется отдельный поток, чтобы можно было обработать сообщения всех пользователей (См. Рисунок 14).

```
def start_server(): 1 usage
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('localhost', 12345))
    server_socket.listen()

    while True:
        client_socket, client_address = server_socket.accept()

        client_thread = threading.Thread(target=try_add_user, args=(client_socket, client_address))
        client_thread.start()
```

Рисунок 14 – Вывод клиента



На стороне клиента также запускается второй поток для чтения из сокета, чтобы не блокировалась возможность писать (См. рисунок 15–16).

```
def receive(client_socket): 1 usage
    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if message:
                print(message)
            else:
                break
        except:
            print("Ошибка")
            client_socket.close()
            break
```

Рисунок 15 – Код клиента

```
def start(): 1 usage
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(('localhost', 12345))

    receive_thread = threading.Thread(target=receive, args=(client_socket,))
    receive_thread.start()

    while True:
        message = input()
        if message.lower() == 'q':
            break
        client_socket.send(message.encode('utf-8'))

    client_socket.close()
```

Рисунок 16 – Код клиента

На рисунке 17–18 представлен пример работы чата при подключении двух человек.

```
Введите свое имя:
danila
natasha присоединился к чату
Привет я Данила
natasha: Привет я Наташа
```

Рисунок 17 – Код клиента

```
Введите свое имя:
natasha
danila: Привет я Данила
Привет я Наташа
```

Рисунок 18 – Код клиента

## **Вывод**

Я изучил работу клиент-серверных приложений и научился работать с сокетами.